

JAVA PROGRAMMING LABORATORY

Experiment No.	06
Date	23/09/26
Title	Implementation of HashMap Methods in Java

Objective / Aim

To write and execute a Java program that demonstrates commonly used methods of the HashMap class, including put(), get(), containsKey(), containsValue(), getOrDefault(), keySet(), values(), entrySet(), size(), remove(), isEmpty(), and clear().

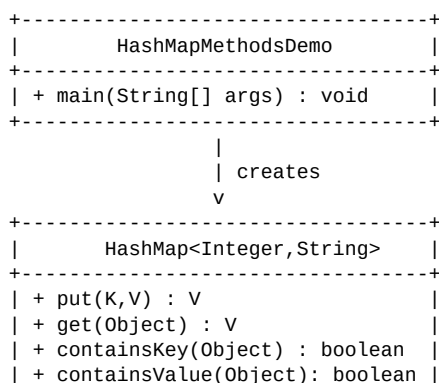
System Requirements

Hardware Requirements	Software Requirements
Computer/Laptop	Operating System: Windows/Linux/macOS
Processor: Intel Core i3 or above	JDK 17 or JDK 21
RAM: 4 GB or above	Java compiler (javac)
Storage: At least 1 GB free space	IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code

Algorithm / Logic Description

1. Import the HashMap class from the java.util package.
2. Create a HashMap named students.
3. Add four student ID and name pairs using put().
4. Use get() to retrieve the student associated with key 102.
5. Use containsKey() to check whether key 103 exists.
6. Use containsValue() to check whether the value Arjun exists.
7. Use getOrDefault() to retrieve key 105 or display Not Found.
8. Use keySet(), values(), and entrySet() to display keys, values, and key-value pairs.
9. Use size() to display the number of mappings.
10. Use remove(103) to remove the mapping with key 103.
11. Use isEmpty() to check whether the HashMap is empty.
12. Use clear() to remove all mappings and check isEmpty() again.

UML Class Diagram



```

| + getOrDefault(Object,V) : V      |
| + keySet() : Set<K>              |
| + values() : Collection<V>       |
| + entrySet() : Set<Entry<K,V>>   |
| + size() : int                   |
| + remove(Object) : V             |
| + isEmpty() : boolean            |
| + clear() : void                 |
+-----+

```

Source Code

```

import java.util.HashMap;

class HashMapMethodsDemo {

    public static void main(String[] args) {

        // Creating a HashMap
        HashMap<Integer, String> students = new HashMap<>();

        // put(K key, V value)
        students.put(101, "Rahul");
        students.put(102, "Priya");
        students.put(103, "Arjun");
        students.put(104, "Sneha");

        System.out.println("After put(): " + students);

        // get(Object key)
        System.out.println("Student with ID 102: " + students.get(102));

        // containsKey(Object key)
        System.out.println("Contains key 103? "
            + students.containsKey(103));

        // containsValue(Object value)
        System.out.println("Contains value Arjun? "
            + students.containsValue("Arjun"));

        // getOrDefault(Object key, V defaultValue)
        System.out.println("Student with ID 105: "
            + students.getOrDefault(105, "Not Found"));

        // keySet()
        System.out.println("Keys: " + students.keySet());

        // values()
        System.out.println("Values: " + students.values());

        // entrySet()
        System.out.println("Key-Value pairs: " + students.entrySet());

        // size()
        System.out.println("Size of HashMap: " + students.size());

        // remove(Object key)
        students.remove(103);
        System.out.println("After remove(103): " + students);

        // isEmpty()
        System.out.println("Is HashMap empty? " + students.isEmpty());

        // clear()
        students.clear();
        System.out.println("After clear(): " + students);

        // isEmpty() after clear
        System.out.println("Is HashMap empty now? " + students.isEmpty());
    }
}

```

Completion and Execution Commands

Save as:
HashMapMethodsDemo.java

Compile:
javac HashMapMethodsDemo.java

```
Execute:  
java HashMapMethodsDemo
```

Output

```
After put(): {101=Rahul, 102=Priya, 103=Arjun, 104=Sneha}  
Student with ID 102: Priya  
Contains key 103? true  
Contains value Arjun? true  
Student with ID 105: Not Found  
Keys: [101, 102, 103, 104]  
Values: [Rahul, Priya, Arjun, Sneha]  
Key-Value pairs: [101=Rahul, 102=Priya, 103=Arjun, 104=Sneha]  
Size of HashMap: 4  
After remove(103): {101=Rahul, 102=Priya, 104=Sneha}  
Is HashMap empty? false  
After clear(): {}  
Is HashMap empty now? true
```

Note: HashMap stores key-value pairs and does not guarantee a fixed iteration order. Therefore, the printed order of entries, keys, and values may vary.